DISTRIBUTED SYSTEMS · ARCHITECTURE & BUILD SPECIFICATION

SolariNet Monitoring

A three-tier, fault-tolerant network & service monitoring system in portable C99

DOCUMENT	VERSION
Architecture & Implementation Plan	1.0 · Architecture reference · 2026-10-20
AUTHOR	**LANGUAGE**
Jason C. Kay	ANSI/ISO C99, K&R-style, functions only
AUDIENCE	**STATUS**
Operator and engineering teams	Implementation built; stabilization gates open

§ 0

How to use this document

This is SolariNet's architecture and implementation reference. The main codebase and several subprojects now exist; this document records the intended system contracts and design decisions.

> CURRENT STATUS · 2026-10-20
> The original phase breakdown below is a historical build plan, not the current release checklist. Stabilization, integration, scanner-ingest I5, verification, and release gates live in the active workflow and the current baseline. A passing local build does not close those gates.

The document proceeds from **strategy** (why) to **architecture** (shape) to **scaffolding** (headers, signatures, schema, build files) to the original **phased task plan**. Use the active workflow for remaining work and the baseline for verified state.

> AGENT INSTRUCTIONS
> Keep the monitoring server core strictly in **C99** using functions and plain `struct`s — no C++, no object orientation, and no opaque OOP emulation beyond the documented handle pattern. Honor §17 coding conventions. The dashboard uses PHP 8.x; Python supports operational automation and standalone integrations. Do not introduce Node.js, server-side JavaScript, or an npm build tool into components that do not already use one.

The dashboard's original visual and data-contract intent is included here; the implemented UI, APIs, SoR, RackWire boundary, and subproject integration are tracked and verified separately in the active workflow.

CONTENTS

1. Strategy & design tenets
2. System architecture
3. Technology decisions
4. Repository layout
5. The SolariNet Control Protocol (SCP)
6. Shared library — libsolari
7. Component: Client
8. Component: Remote Monitor
9. Component: Server
10. Database schema
11. Dashboard data contract
12. Build system & cross-compilation
13. Configuration formats
14. Testing strategy
15. Deployment & operations
16. Phased project plan & task breakdown
17. Coding standards
18. Appendices

§ 1

Strategy & design tenets

SolariNet observes a private network (an intranet of roughly 60 devices) from three vantage points and fuses their views into one authoritative, continuously-published picture of health.

The three vantage points

Each tier answers a different question, and no single tier can answer all three — which is precisely why the system is distributed:

TIER	VANTAGE	ANSWERS	VOLUME
Client `solariClient`	Inside the host, looking down	"Is *this machine* healthy? CPU, RAM, swap, disk, NIC/USB throughput, key processes, their open files & sockets, log files."	One per host, plus a watchdog
Remote Monitor `solariMonitor`	On the wire, looking across	"Are services *reachable and fast* from where I stand? TCP/UDP/ICMP probes, latency, transit time, dropped packets, throughput vs. capacity, link characteristics, peer-to-peer reachability."	~10% of nodes, at varied vantage points
Server `solariServer`	Above all, fusing	"What is the *whole-network* state, historically and now? Collects client + monitor data, owns master records, publishes the C2 dashboard, manages config & deployment of every node."	One primary, one failover

Design tenets

1. **Portability is a first-class requirement, not a retrofit.** The client must build for Windows, macOS, and Linux across ARM32/ARM64/x86/x86_64. All OS-specific code lives behind a single platform-abstraction header (`platOS.h`); the core never calls a syscall directly.
2. **Fault tolerance by construction.** Monitors are interchangeable and redundant — losing one degrades coverage, never correctness. The server runs primary/failover with a database-mediated lease, so a primary failure is a sub-30-second promotion, not an outage.
3. **One wire protocol, versioned from day one.** Every tier-to-tier message rides the SolariNet Control Protocol (§5): a versioned, length-prefixed frame with a binary header and TLV payload. New fields never break old peers.
4. **The C backend never renders HTML.** The server writes MariaDB; PHP-on-Apache reads it. This keeps the C strictly to measurement, transport, and persistence, and lets the dashboard evolve independently.
5. **Local control, no cloud dependency.** Every component runs on-prem. No telemetry leaves the intranet. TLS and node enrollment are self-managed.
6. **Functions, not objects.** Plain C, K&R lineage. State travels in explicit context structs passed by pointer (the "handle" pattern, §17) — never hidden in globals, never wrapped in pseudo-classes.

Non-goals

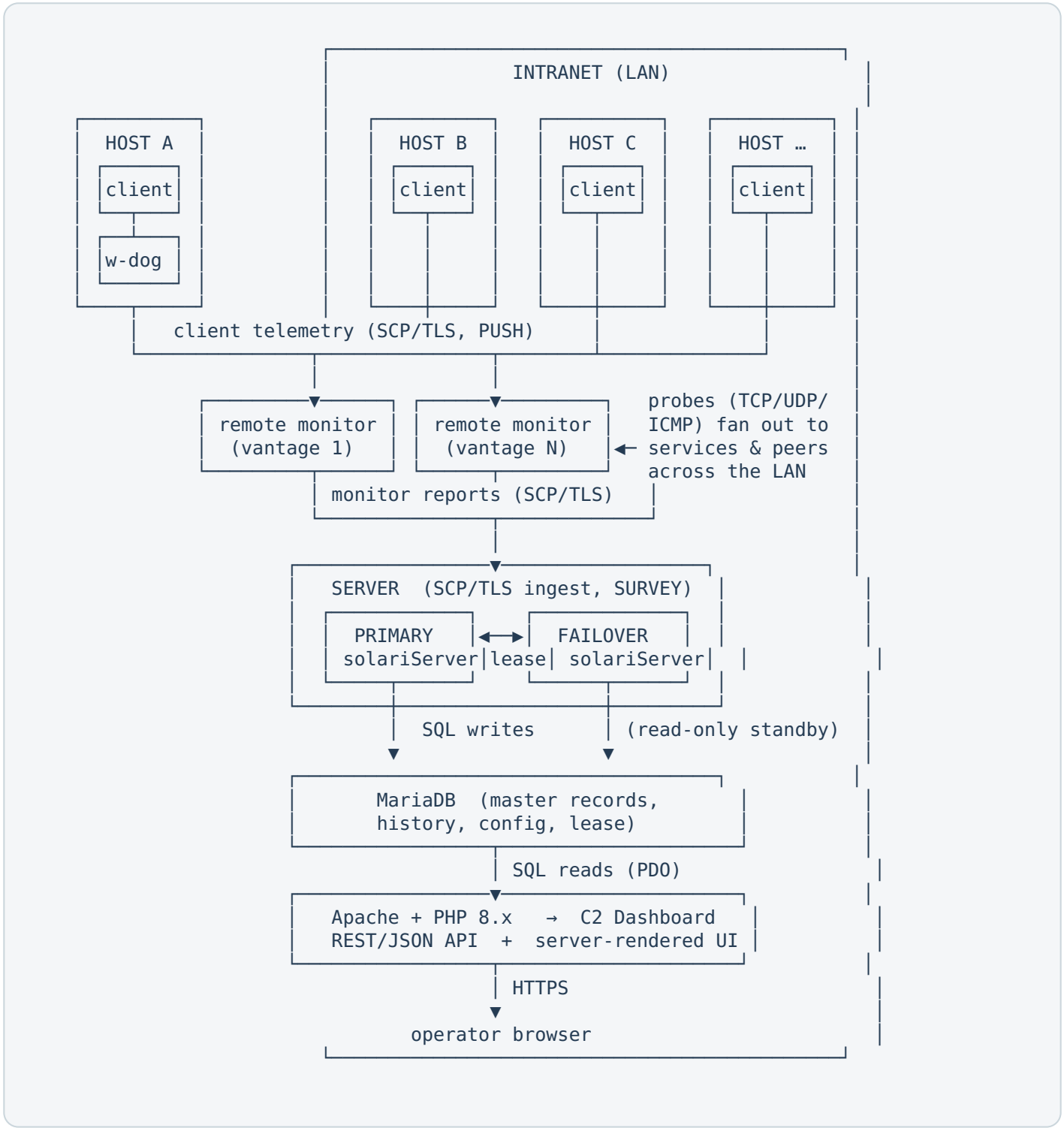
- **Not** a public-internet, multi-tenant SaaS. Single-tenant, single intranet, trusted-ish LAN with TLS for defense in depth.
- **Not** a metrics time-series database competitor (Prometheus et al. already exist in the environment). SolariNet's edge is the *fused three-vantage view* and the C2 control plane, not raw TSDB throughput.
- **Not** a config-management replacement. The server's "manage config/deployment" role pushes *SolariNet's own* node config and binaries, not general-purpose CM.

---

§ 2

System architecture

Three C programs, one shared library, one relational store, one PHP dashboard. Data flows up; control flows down.



Data flow

1. **Client → Server (push).** Each client samples local metrics on a schedule (default 15 s) and pushes an `SCP_MSG_CLIENT_REPORT` frame to the server's ingest endpoint. Watchdog supervises the client process and emits `SCP_MSG_WATCHDOG` heartbeats independently.
2. **Monitor → Server (push + survey).** Monitors run their probe schedule autonomously and push `SCP_MSG_MONITOR_REPORT` frames. The server can also actively *survey* the monitor fleet (nng SURVEYOR/RESPONDENT) to demand an immediate round, e.g. on operator request from the dashboard.
3. **Server → MariaDB.** The active server validates, deduplicates, upserts master records, appends history rows, and evaluates alert rules.
4. **Dashboard → MariaDB.** PHP reads via PDO. The C server and PHP never talk directly; the database is the contract boundary.
5. **Server → nodes (control, down).** Config changes and binary/deploy directives flow as `SCP_MSG_CONTROL` frames addressed to specific nodes.

Failover model

Two server instances, exactly one active. Coordination is database-mediated to avoid pulling in a consensus library for a two-node problem:

- A single row in the `serverLease` table holds `(leaseHolder, leaseEpoch, expiresAt)`.
- The active server renews the lease every `leaseRenewSec` (default 5 s) via a conditional `UPDATE … WHERE expiresAt < NOW() OR leaseHolder = me` inside a transaction. The atomicity of that single-row update *is* the election.
- The standby polls the lease. If it observes `expiresAt < NOW()`, it attempts to claim it with an incremented `leaseEpoch`. Winner becomes active and begins binding ingest sockets; loser backs off.
- Nodes always know both server addresses (from config); they connect to whichever currently holds the lease, discovered via a tiny `SCP_MSG_WHO_IS_PRIMARY` query answered by either instance.

> WHY DB-LEASE, NOT RAFT
>
> For N=2 servers sharing a MariaDB they already depend on, a leased single-row mutex gives correct single-writer semantics with zero extra dependencies and trivial debuggability. Raft/etcd would add an operational component to babysit for no benefit at this cardinality. Split-brain is bounded by lease TTL and made harmless because all writes funnel through the same row's conditional update. *If* the fleet ever needs ≥3 active coordinators, revisit — but that is explicitly a non-goal (§1).

Security model

- **Transport:** All SCP traffic is TLS 1.2+ via nng's TLS transport backed by **mbedTLS**. Plaintext SCP is permitted only on loopback (client ↔ its own watchdog) and only when explicitly configured.
- **Node identity & enrollment:** Each node carries a long-lived X.509 client certificate issued at enrollment. Enrollment is a one-time operator-approved exchange: a node presents a pre-shared enrollment token (out-of-band), the server signs its CSR, and the node thereafter authenticates by cert. This dovetails with the operator's existing internal CA (step-ca) — the server's trust store is simply the internal CA root.
- **Authorization:** A node's certificate CN encodes its role (`client` / `monitor` / `server`); the server rejects frames whose type is illegal for the presenting role (a client may not send `SCP_MSG_MONITOR_REPORT`, etc.).
- **Dashboard:** Operator auth at the PHP layer (session + RBAC). The dashboard is read-mostly; any control action (trigger survey, push config) is POSTed to the REST API, authenticated, and translated by a small privileged helper into an `SCP_MSG_CONTROL` to the server. The PHP layer never holds a server certificate directly — it calls a local `solariCtl` Unix-socket helper that does.

---

§ 3

Technology decisions

Every dependency is justified, license-checked for linkage, and chosen for portability across the full architecture matrix. Rejected alternatives are recorded so the agent doesn't re-litigate them.

CONCERN	CHOICE	LICENSE	RATIONALE & REJECTED ALTERNATIVES
Core language	ISO C99	—	Mandated. C99 (not C89) for `<stdint.h>`, `<stdbool.h>`, designated initializers, and mixed declarations — all portable across every target compiler (GCC, Clang, MSVC in C mode). K&R brace style, functions only.
Tier-to-tier messaging	**nng** (nanomsg-next-gen)	`MPL-2.0`	Single, portable C library implementing the exact scalability patterns we need: `PUSH/PULL` (best-effort ingest), `REQ/REP` (control plus durable-report ACKs), `SURVEYOR/RESPONDENT` (server polls monitor fleet), and `PUB/SUB` (server → dashboards/notifications). Handles reconnect, backpressure, and message framing so we don't hand-roll it badly. MPL-2.0 is file-level copyleft — links cleanly into our C without infecting our source. *Rejected:* ZeroMQ (LGPL + C++ core, heavier), raw sockets for the control plane (we'd reinvent nng's reconnection/poll logic), gRPC (drags in C++/protobuf, violates the no-heavy-runtime tenet).
TLS	**mbedTLS**	`Apache-2.0`	Small, portable, builds cleanly for embedded ARM through x86_64; integrates as nng's TLS engine. Apache-2.0 links freely. *Rejected:* OpenSSL (heavier, thornier cross-builds on Windows/macOS for our needs), though it remains a documented fallback if a target platform ships it more readily.
Raw service probing	Hand-rolled BSD sockets + raw ICMP	—	The monitor's *measurement surface* (latency, packet loss, TCP connect timing) needs direct socket control nng can't give. Kept in portable C behind `probeNet.h`; ICMP's raw-socket / privilege quirks are isolated per-platform (see Appendix C).
Persistence	**MariaDB** 10.6+	`GPL-2 (server) / LGPL (connector-c)`	Operator standard; the C server links **MariaDB Connector/C** (LGPL-2.1, links fine). The DB is the contract boundary between C and PHP. *Rejected:* embedding SQLite + serving the dashboard from C (couples measurement to presentation, contradicts tenet 4; SQLite stays as the optional local spool only — see below).
Local spool / store-and-forward	**SQLite** (amalgamation)	`Public domain`	Single-file amalgamation compiled directly in. Each client/monitor retains durable reports in SQLite until the authenticated server ACKs the matching sequence and reporter session; reconnect replay is deduplicated through transactional receipts. Public domain, zero license friction.
Dashboard runtime	**PHP 8.x on Apache**	—	Operator standard; reads MariaDB via PDO. Server-rendered UI + JSON REST API. *Explicitly no Node/SSR-JS.* Browser-side JS is fine and expected for the live dashboard.
Build system	**CMake** ≥3.16 + thin Makefile wrapper	`BSD`	The only practical cross-platform/cross-arch C build orchestrator (toolchain files per target, §12). A top-level `Makefile` wraps common invocations for muscle-memory (`make client`, `make monitor`, `make cross-arm64`). *Rejected:* Autotools (painful on Windows/MSVC), bare Make (no clean cross-toolchain story).
Unit testing	**Unity** + CMock	`MIT`	Tiny pure-C test framework (single .c/.h), perfect for embedded-grade portability. Drives §14's unit suites. *Rejected:* heavier C++-based frameworks (Catch2/GoogleTest) — wrong language, wrong weight.
JSON (where C must emit/parse)	**cJSON**	`MIT`	The server emits a JSON projection of SCP for any path the dashboard/API consumes directly, and parses JSON config overlays. Single-file, MIT, trivially portable. *Rejected:* Jansson (fine but heavier), hand-rolled (error-prone).

DEPENDENCY VENDORING

All third-party C deps (nng, mbedTLS, SQLite amalgamation, cJSON, Unity) are vendored under `third_party/` as pinned source or git submodules and built from source by CMake. No system-package assumptions, no package manager at build time. This is what makes the cross-architecture matrix tractable and the build hermetic. MariaDB Connector/C is the one exception: linked against the platform's dev package on server targets (x86_64/ARM64 Linux only), since the server is not cross-compiled to exotic targets.

---

§ 4

Repository layout

A single git monorepo. Shared code is real, versioned, and built once into `libsolari`; each component links it.

```
solarinet/
├── CMakeLists.txt              # top-level build (§12)
├── Makefile                    # convenience wrapper over CMake
├── README.html                 # self-contained project README (not .md)
├── LICENSE
├── cmake/
│   ├── toolchains/             # cross-compile toolchain files
│   │   ├── linux-arm32.cmake
│   │   ├── linux-arm64.cmake
│   │   ├── linux-x86_64.cmake
│   │   ├── windows-x86_64.cmake   # mingw-w64
│   │   └── macos-universal.cmake
│   └── modules/                # Find*.cmake helpers
├── include/solari/             # PUBLIC headers for libsolari (§6)
│   ├── solariCommon.h
│   ├── solariFrame.h           # SCP framing
│   ├── solariTlv.h             # TLV encode/decode
│   ├── solariMsg.h             # message type defs & structs
│   ├── solariConfig.h
│   ├── solariLog.h
│   ├── solariTime.h
│   ├── solariNet.h             # nng transport helpers
│   ├── solariCrypto.h          # hashing, cert helpers
│   ├── solariSpool.h           # SQLite store-and-forward
│   └── solariError.h
├── lib/solari/                 # libsolari implementation
│   └── *.c
├── src/
│   ├── client/                 # solariClient (§7)
│   │   ├── main.c
│   │   ├── clientCollect.c
│   │   ├── clientReport.c
│   │   ├── clientWatchdog.c
│   │   ├── platOS.h            # platform abstraction (the portability seam)
│   │   └── plat/
│   │       ├── platLinux.c
│   │       ├── platMacos.c
│   │       └── platWindows.c
│   ├── monitor/                # solariMonitor (§8)
│   │   ├── main.c
│   │   ├── probeNet.c          # tcp/udp/icmp probes
│   │   ├── probeNet.h
│   │   ├── monitorSched.c
│   │   ├── monitorPeer.c       # peer mesh / redundancy
│   │   └── monitorReport.c
│   └── server/                 # solariServer (§9)
│       ├── main.c
│       ├── serverIngest.c
│       ├── serverLease.c       # failover election
│       ├── serverDb.c          # MariaDB writer
│       ├── serverMaster.c      # master-record reconciliation
│       ├── serverControl.c     # config/deploy push
│       ├── serverAlert.c       # rule evaluation
│       └── solariCtl.c         # privileged unix-socket helper for PHP
├── dashboard/                  # PHP 8.x + Apache (§11) — contract only here
│   ├── public/                 # docroot
│   ├── api/                    # REST/JSON endpoints
│   ├── lib/                    # PDO data-access layer
│   └── config/
├── db/
│   ├── schema.sql              # MariaDB DDL (§10)
│   ├── seed.sql
│   └── migrations/
├── config/                     # sample .conf files (§13)
│   ├── client.conf.sample
│   ├── monitor.conf.sample
│   └── server.conf.sample
├── deploy/
│   ├── systemd/                # unit files (§15)
│   └── enrollment/             # enrollment scripts
├── tests/
│   ├── unit/                   # Unity suites (§14)
│   ├── integration/
│   └── fault/                  # fault-injection harnesses
├── third_party/                # vendored deps (pinned)
```

```
│   ├── nng/  mbedtls/  sqlite/  cJSON/  unity/
└── docs/                       # self-contained HTML docs
```

---

§ 5

The SolariNet Control Protocol (SCP)

One versioned, length-prefixed binary protocol carries every tier-to-tier message. A canonical JSON projection exists for any consumer that reads SCP data without speaking it natively (the dashboard, debugging tools).

5.1 Framing

Every SCP message on the wire is a single frame. nng delivers messages atomically, so the outer length prefix is redundant *over nng* but is retained so the format is self-delimiting over raw streams (spool files, loopback plaintext, packet captures).

```
  /* All multi-byte integers are big-endian (network byte order). */

  +----------------+----------------------+--------------------------+--------+
  | uint32 frameLen| FrameHeader (32 bytes)| TLV payload (frameLen-32)| uint32 |
  | (payload bytes)|                       |                          |  crc32 |
  +----------------+----------------------+--------------------------+--------+

  frameLen   = byte count of [FrameHeader + TLV payload]  (excludes itself & trailing crc32)
  crc32      = CRC-32 (IEEE) over [FrameHeader + TLV payload], integrity check only
```

5.2 Frame header (fixed 32 bytes)

▸ include/solari/solariFrame.h — wire header

```
typedef struct {
    uint8_t  magic[2];      /* {'S','N'} — SolariNet sentinel             */
    uint8_t  protoVersion;   /* SCP_PROTO_VERSION; bump only on breaking chg */
    uint8_t  msgType;        /* enum solariMsgType (5.4)                   */
    uint8_t  flags;          /* SCP_FLAG_* bitfield (5.3)                  */
    uint8_t  reserved;       /* 0; keeps following fields aligned          */
    uint16_t tlvCount;       /* number of TLV elements in payload          */
    uint64_t sourceNodeId;   /* stable 64-bit node id (5.5)                */
    uint64_t sendTimeUnixMs; /* sender clock, ms since epoch               */
    uint32_t seqNo;          /* per-source monotonic; dedup & ordering      */
    uint32_t correlationId;  /* echoes a request's seqNo in its reply; 0 if  */
                             /* unsolicited                                 */
} solariFrameHeader;         /* sizeof == 32, statically asserted          */
```

> IMPLEMENTATION NOTE
>
> Do not `memcpy` the struct onto the wire — pack/unpack field-by-field through the accessor functions in §6 so endianness and alignment are correct on every architecture. A `SOLARI_STATIC_ASSERT(sizeof(solariFrameHeader)==32)` guards the layout, but the wire form is defined by the pack functions, not the struct memory image.

5.3 Flags

FLAG	VALUE	MEANING
`SCP_FLAG_NONE`	0x00	No flags.
`SCP_FLAG_ACK_REQ`	0x01	Sender wants an `SCP_MSG_ACK` with matching `correlationId`. Durable reports also require the ACK TLV to echo `TLV_REPORT_SESSION`; the server ACKs only after report persistence.
`SCP_FLAG_COMPRESSED`	0x02	Reserved for DEFLATE payloads; protocol v1 receivers reject it because compression is not implemented.
`SCP_FLAG_SPOOL_REPLAY`	0x04	Frame is a store-and-forward replay; server should dedup by `(sourceNodeId, seqNo)`.
`SCP_FLAG_URGENT`	0x08	Bypass normal batching; evaluate alert rules immediately.

Protocol v1 receivers require the reserved header byte to be zero, reject unsupported flag bits, and verify that the payload is a complete TLV sequence whose element count matches `tlvCount`. Unknown TLV types remain forward-compatible and count as elements.

5.4 Message types

TYPE	CODE	DIRECTION	PURPOSE
`SCP_MSG_HELLO`	0x01	node → server	Session open; declares role, version, capabilities.
`SCP_MSG_WELCOME`	0x02	server → node	Accepts session; returns assigned schedule & config epoch.
`SCP_MSG_CLIENT_REPORT`	0x10	client ↔ server	Local host metrics snapshot. ACK_REQ uses the authenticated control REQ/REP endpoint; ACK follows durable receipt and report persistence.
`SCP_MSG_MONITOR_REPORT`	0x11	monitor ↔ server	Probe results batch. ACK_REQ uses the authenticated control REQ/REP endpoint; ACK follows durable receipt and report persistence.
`SCP_MSG_WATCHDOG`	0x12	watchdog → server	Liveness heartbeat for a supervised client.
`SCP_MSG_SURVEY`	0x20	server → fleet	SURVEYOR: demand an immediate report round.
`SCP_MSG_SURVEY_RESP`	0x21	node ↔ server	RESPONDENT report; ACK_REQ uses the authenticated control REQ/REP endpoint and follows durable persistence.
`SCP_MSG_CONTROL`	0x30	server → node	Config change / deploy directive (5.8).
`SCP_MSG_CONTROL_RESULT`	0x31	node → server	Outcome of a control directive.
`SCP_MSG_WHO_IS_PRIMARY`	0x40	node → server	Failover discovery query.
`SCP_MSG_PRIMARY_IS`	0x41	server → node	Names the current lease holder.
`SCP_MSG_ACK`	0x50	server → node	Durable-report acknowledgment. Correlation echoes report seqNo; `TLV_REPORT_SESSION` echoes the reporter session.
`SCP_MSG_ERROR`	0x51	any → any	Structured error (code in TLV, see Appendix A).

5.5 Node identity

Decision — retain the deployed deterministic node ID contract. Canonical node IDs use `FNV-1a-64("hostFqdn|roleNumber")`, implemented by `solariNodeIdFromFqdnRole()`. A read-only production query on 2026-10-21 confirmed that all nine deployed node IDs match this formula. The proposed nonce input has no established lifecycle or persisted value in the current enrollment system; introducing it would change IDs and require a coordinated migration. Remove the nonce from the architecture target unless a future design defines that lifecycle and a compatibility migration. Node IDs remain identifiers, not credentials: mutual TLS authenticates peers and server-side identity checks bind them to the enrolled node.

Certificate name migration remains open. The same production query found all nine database `certCn` values are host-only, while the candidate enrollment and ingest contract now requires role-prefixed names such as `monitor.hostFqdn`. This is a database observation, not independent validation of each certificate's subject or SAN. Do not deploy the stricter candidate path until existing certificates are re-enrolled or a reviewed compatibility transition is in place. Preserve role-prefixed CN enforcement for new enrollment and authenticated frame binding.

5.6 TLV payload

Payloads are sequences of Type-Length-Value triples. This is what lets the protocol evolve without version bumps: unknown TLV types are skipped, not fatal.

▸ include/solari/solariTlv.h — element layout

```
  +----------+----------+------------------+
  | uint16   | uint16   | value            |
  | tlvType  | tlvLen   | (tlvLen bytes)   |
  +----------+----------+------------------+

/* TLV type ranges (high byte = category):                        */
/*   0x01xx  identity & session   0x10xx  host metrics            */
/*   0x11xx  probe results        0x30xx  control directives      */
/*   0xFFxx  error / diagnostic                                   */

enum solariTlvType {
    /* --- identity / session (0x01xx) --- */
    TLV_ROLE               = 0x0101,  /* uint8: 1=client 2=monitor 3=server   */
    TLV_AGENT_VERSION      = 0x0102,  /* utf8 string                          */
    TLV_HOST_FQDN          = 0x0103,  /* utf8 string                          */
    TLV_CONFIG_EPOCH       = 0x0104,  /* uint64                               */
    /* --- host metrics (0x10xx) --- */
    TLV_OS_NAME            = 0x1001,  /* utf8: "Linux 6.1.0", "macOS 14.5"…   */
    TLV_ARCH               = 0x1002,  /* utf8: "arm64","x86_64","armv7","ppc" */
    TLV_CPU_LOAD_MILLI     = 0x1003,  /* uint32: load%*1000, per-core array   */
    TLV_RAM_USED_KB        = 0x1004,  /* uint64                               */
    TLV_RAM_TOTAL_KB       = 0x1005,  /* uint64                               */
    TLV_SWAP_USED_KB       = 0x1006,  /* uint64                               */
    TLV_SWAP_TOTAL_KB      = 0x1007,  /* uint64                               */
    TLV_DISK_FREE_KB       = 0x1008,  /* repeated: {mountStr, uint64 free}    */
    TLV_NET_IFACE          = 0x1009,  /* repeated: iface throughput vs cap    */
    TLV_USB_THROUGHPUT     = 0x100A,  /* repeated: bus throughput vs capacity */
    TLV_PROC_WATCH         = 0x100B,  /* repeated: {name, pid, state, nFiles, */
                                      /*            nSockets, rssKb}          */
    TLV_LOGFILE_STAT       = 0x100C,  /* repeated: {path, sizeDelta, matchCt} */
    TLV_NET_ADDR           = 0x100D,  /* repeated: interface IP/MAC           */
    /* --- probe results (0x11xx) --- */
    TLV_PROBE_RESULT       = 0x1101,  /* repeated, struct in 5.7              */
    TLV_LINK_STAT          = 0x1102,  /* repeated: latency/loss/jitter per pr */
    TLV_SERVICE_META       = 0x1103,  /* repeated: version/vendor/config kv   */
    /* --- control (0x30xx) --- */
    TLV_CTRL_VERB          = 0x3001,  /* uint8: setConfig|deploy|restart|...  */
    TLV_CTRL_PAYLOAD       = 0x3002,  /* opaque blob (e.g. new config, hash)  */
    TLV_CTRL_TARGET_EPOCH  = 0x3003,  /* uint64 config epoch to converge to   */
    /* --- lifecycle (0x32xx) --- */
    TLV_LIFE_CONFIRM       = 0x3201,  /* uint64 one-time operator challenge    */
    TLV_LIFE_WIPE_SCOPE    = 0x3202,  /* uint8 selected teardown scope         */
    TLV_LIFE_GENERATION    = 0x3203,  /* uint64 generation-bound ack identity  */
    /* --- error (0xFFxx) --- */
    TLV_ERROR_CODE         = 0xFF01,  /* uint16, Appendix A                   */
    TLV_ERROR_DETAIL       = 0xFF02   /* utf8 string                          */
};
```

5.7 Probe-result value layout (TLV_PROBE_RESULT)

```
typedef struct {
    uint8_t  proto;        /* 1=ICMP 2=TCP 3=UDP                      */
    uint8_t  outcome;      /* enum probeOutcome: ok/timeout/refused/… */
    uint16_t dstPort;      /* 0 for ICMP                              */
    uint8_t  dstAddr[16];  /* IPv4-mapped or IPv6                     */
    uint32_t rttMicros;    /* round-trip time, microseconds           */
    uint32_t jitterMicros; /* inter-probe variance                    */
    uint16_t lossPermille; /* packets lost per thousand               */
    uint32_t throughputKbps;
    uint64_t probeTimeUnixMs;
} solariProbeResult;     /* packed/unpacked via accessors, never raw memcpy */
```

5.8 Control directives (TLV_CTRL_VERB)

VERB	VALUE	EFFECT ON NODE
`CTRL_SET_CONFIG`	1	Apply config blob in `TLV_CTRL_PAYLOAD`, converge to `TARGET_EPOCH`, reply with result.
`CTRL_DEPLOY`	2	Fetch/verify (by hash in payload) and stage a new agent binary; await `CTRL_RESTART`.
`CTRL_RESTART`	3	Graceful self-restart (watchdog re-execs).
`CTRL_PROBE_NOW`	4	(Monitor) run probe round immediately, out of schedule.
`CTRL_SET_SCHEDULE`	5	Change sampling/probe interval.
`CTRL_DRAIN_SPOOL`	6	Flush local store-and-forward spool now.

5.9 Versioning rules

- **Adding a TLV type** → no version bump. Old peers skip unknown TLVs.
- **Adding a message type** → no version bump if unsolicited and ignorable; receivers reply `SCP_MSG_ERROR/ERR_UNKNOWN_MSG` for types they must understand.
- **Changing header layout or an existing TLV's semantics** → bump `SCP_PROTO_VERSION`. The server supports the current and previous major version during rollout.

5.10 Canonical JSON projection

The server can serialize any SCP message to a stable JSON form (via cJSON) for the dashboard/API and for debugging. Field names mirror TLV names in camelCase. This projection is *read-only*: nothing parses JSON back into SCP frames except test fixtures.

```
{
  "header": {
    "protoVersion": 1, "msgType": "CLIENT_REPORT",
    "sourceNodeId": "0x9f3a…", "sendTimeUnixMs": 1733500000123,
    "seqNo": 4412, "flags": ["URGENT"]
  },
  "hostFqdn": "hydrogen.akoria.net",
  "osName": "Linux 6.1.0", "arch": "arm64",
  "cpuLoadMilli": [320,410,290,380],
  "ramUsedKb": 5242880, "ramTotalKb": 16777216,
  "procWatch": [
    { "name": "apache2", "pid": 812, "state": "R",
      "nFiles": 142, "nSockets": 38, "rssKb": 88210 }
  ]
}
```

§ 6

Shared library — `libsolari`

Everything common to all three components lives here, built once. The signatures below *are the contract*: where no body is shown, the doc-comment defines correct behavior and the agent implements to it.

6.1 Common types & conventions

▸ include/solari/solariCommon.h

```c
/*
 * solariCommon.h — shared types, return conventions, static-assert macro.
 * All libsolari functions return solariStatus; SOLARI_OK (0) is success,
 * negative values are errors enumerated in solariError.h. Out-params are
 * written only on success unless documented otherwise.
 */
#ifndef SOLARI_COMMON_H
#define SOLARI_COMMON_H
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

typedef int32_t solariStatus;   /* SOLARI_OK or negative solariError code */

#define SOLARI_OK 0
#define SCP_PROTO_VERSION 1
#define SOLARI_MAX_FRAME (1u << 20)   /* 1 MiB hard cap on a single frame */

/* Compile-time assertion usable in C99 (no _Static_assert dependency). */
#define SOLARI_STATIC_ASSERT(cond, tag) \
    typedef char solariStaticAssert_##tag[(cond) ? 1 : -1]

typedef enum { ROLE_CLIENT = 1, ROLE_MONITOR = 2, ROLE_SERVER = 3 } solariRole;

#endif
```

6.2 Frame pack / unpack

▸ include/solari/solariFrame.h

```c
/* Pack a header into buf (must be >= 32 bytes), big-endian, field by field.
 * Returns SOLARI_OK; never touches endianness of the caller's struct.      */
solariStatus solariFramePackHeader(const solariFrameHeader *hdr,
                                   uint8_t *buf, size_t bufLen);

/* Inverse of the above. Validates magic and protoVersion; on version
 * mismatch returns ERR_PROTO_VERSION but still fills hdr for diagnostics.   */
solariStatus solariFrameUnpackHeader(const uint8_t *buf, size_t bufLen,
                                     solariFrameHeader *hdr);

/* Assemble a complete frame (len prefix + header + payload + crc32) into
 * out. *outLen is set to bytes written. Computes crc32 internally.          */
solariStatus solariFrameBuild(const solariFrameHeader *hdr,
                              const uint8_t *payload, size_t payloadLen,
                              uint8_t *out, size_t outCap, size_t *outLen);

/* Parse one frame from a byte stream. Verifies crc32. On success, sets
 * hdr and points *payload into buf (no copy) with *payloadLen. Returns
 * ERR_FRAME_INCOMPLETE if buf holds less than one full frame.              */
solariStatus solariFrameParse(const uint8_t *buf, size_t bufLen,
                              solariFrameHeader *hdr,
                              const uint8_t **payload, size_t *payloadLen,
                              size_t *frameTotalLen);
```

6.3 TLV encode / decode

▸ include/solari/solariTlv.h

```c
/* A growable write cursor over a caller-owned buffer. Functions style
 * is fluent-by-status: each append returns SOLARI_OK or ERR_BUFFER_FULL. */
typedef struct {
    uint8_t *buf; size_t cap; size_t len; uint16_t count;
} solariTlvWriter;

void solariTlvWriterInit(solariTlvWriter *w, uint8_t *buf, size_t cap);

solariStatus solariTlvAppend(solariTlvWriter *w, uint16_t type,
                             const void *val, uint16_t len);
solariStatus solariTlvAppendU8 (solariTlvWriter *w, uint16_t type, uint8_t  v);
solariStatus solariTlvAppendU16(solariTlvWriter *w, uint16_t type, uint16_t v);
solariStatus solariTlvAppendU32(solariTlvWriter *w, uint16_t type, uint32_t v);
solariStatus solariTlvAppendU64(solariTlvWriter *w, uint16_t type, uint64_t v);
solariStatus solariTlvAppendStr(solariTlvWriter *w, uint16_t type, const char *s);

/* Read cursor. Iterate with solariTlvNext until ERR_TLV_END. The returned
 * val pointer aliases the source buffer (zero-copy); do not free it.      */
typedef struct { const uint8_t *buf; size_t len; size_t pos; } solariTlvReader;

void solariTlvReaderInit(solariTlvReader *r, const uint8_t *buf, size_t len);
solariStatus solariTlvNext(solariTlvReader *r, uint16_t *type,
                           const uint8_t **val, uint16_t *len);
/* Convenience typed reads that also validate length. */
solariStatus solariTlvReadU32(const uint8_t *val, uint16_t len, uint32_t *out);
solariStatus solariTlvReadU64(const uint8_t *val, uint16_t len, uint64_t *out);
```

6.4 Typed message builders

Above the raw TLV layer, each message type gets a builder/parser pair so components never touch TLV codes directly. One representative pair shown; the rest follow the identical pattern.

▸ include/solari/solariMsg.h (excerpt)

```c
typedef struct {               /* in-memory form of a client report */
    char     hostFqdn[256];
    char     osName[64];
    char     arch[16];
    uint32_t cpuLoadMilli[SOLARI_MAX_CORES];
    uint8_t  coreCount;
    uint64_t ramUsedKb, ramTotalKb, swapUsedKb, swapTotalKb;
    solariDiskEntry  disks[SOLARI_MAX_DISKS];   uint8_t diskCount;
    solariIfaceEntry ifaces[SOLARI_MAX_IFACES]; uint8_t ifaceCount;
    solariProcEntry  procs[SOLARI_MAX_PROCS];   uint8_t procCount;
    solariLogEntry   logs[SOLARI_MAX_LOGS];     uint8_t logCount;
} solariClientReport;

/* Serialize report into a frame payload (TLV). */
solariStatus solariMsgBuildClientReport(const solariClientReport *rep,
                                        uint8_t *payload, size_t cap,
                                        size_t *outLen, uint16_t *tlvCount);
/* Parse a frame payload back into a report (server side). */
solariStatus solariMsgParseClientReport(const uint8_t *payload, size_t len,
                                        solariClientReport *rep);
```

6.5 Transport (nng wrapper)

▸ include/solari/solariNet.h

```c
/* Opaque transport handle. Wraps an nng socket + TLS config. The "handle
 * pattern": state lives in a heap struct the caller threads through calls;
 * there are no globals and no hidden statics. */
typedef struct solariConn solariConn;

typedef enum {
    SOLARI_PATTERN_PUSH,    /* client/monitor -> server ingest      */
    SOLARI_PATTERN_PULL,    /* server ingest sink                    */
    SOLARI_PATTERN_SURVEYOR,/* server -> fleet demand                */
    SOLARI_PATTERN_RESPOND, /* node answers surveys                  */
    SOLARI_PATTERN_PUB,     /* server -> dashboards/notifiers        */
    SOLARI_PATTERN_SUB,
    SOLARI_PATTERN_REQ,     /* who-is-primary, control acks          */
    SOLARI_PATTERN_REP
} solariPattern;

typedef struct {
    const char *url;          /* "tls+tcp://host:port" or "tcp://…"   */
    solariPattern pattern;
    bool        useTls;
    const char *caFile;       /* internal CA root                      */
    const char *certFile;     /* this node's cert                      */
    const char *keyFile;
    int         dialTimeoutMs;
    int         sendTimeoutMs;
} solariConnOpts;

solariStatus solariConnOpen (const solariConnOpts *opts, solariConn **out);
solariStatus solariConnListen(const solariConnOpts *opts, solariConn **out);
/* Send one fully-built frame. Honors send timeout; on transient failure
 * returns ERR_CONN_RETRY so callers know to spool. */
solariStatus solariConnSend (solariConn *c, const uint8_t *frame, size_t len);
/* Receive one frame into a library-managed buffer; *frame valid until next
 * recv on the same conn. Blocks up to recvTimeoutMs. */
solariStatus solariConnRecv (solariConn *c, const uint8_t **frame, size_t *len);
void         solariConnClose(solariConn *c);
```

6.6 Store-and-forward spool

▸ include/solari/solariSpool.h

```c
/* SQLite-backed durable queue. When solariConnSend returns ERR_CONN_RETRY,
 * the caller enqueues the frame here; a drain thread retries on reconnect.
 * This is the mechanism behind monitor/client fault tolerance. */
typedef struct solariSpool solariSpool;

solariStatus solariSpoolOpen  (const char *dbPath, solariSpool **out);
solariStatus solariSpoolPush  (solariSpool *s, const uint8_t *frame, size_t len);
/* Pull the oldest undelivered frame; caller sends, then calls Ack on success
 * or Nack (increments attempt count, applies backoff) on failure. */
solariStatus solariSpoolPeek  (solariSpool *s, uint64_t *rowId,
                               const uint8_t **frame, size_t *len);
solariStatus solariSpoolAck   (solariSpool *s, uint64_t rowId);
solariStatus solariSpoolNack  (solariSpool *s, uint64_t rowId);
size_t       solariSpoolDepth (solariSpool *s);
void         solariSpoolClose(solariSpool *s);
```

6.7 Config, logging, time, crypto

These follow the same handle/return-status conventions. Headers in full in the repo; signatures summarized:

HEADER	KEY FUNCTIONS	NOTES
`solariConfig.h`	`solariConfigLoad()`, `solariConfigGetStr/Int/Bool()`, `solariConfigEpoch()`	Parses the `key = value` .conf format (§13); supports a JSON overlay merged on top. Returns a config handle.
`solariLog.h`	`solariLogInit()`, `solariLogf(level, fmt, …)`, `solariLogSetLevel()`	Five levels (TRACE→FATAL), syslog + file sinks, ISO-8601 UTC timestamps. Thread-safe.
`solariTime.h`	`solariNowUnixMs()`, `solariMonotonicMicros()`, `solariSleepMs()`	Monotonic clock for RTT (never wall clock); wall clock only for timestamps.
`solariCrypto.h`	`solariFnv1a64()`, `solariCrc32()`, `solariSha256()`, `solariCertCn()`	Node-id hashing, frame CRC, deploy-blob verification, cert CN extraction for role auth.
`solariError.h`	`solariStrError(solariStatus)`	Central error enum (Appendix A) + human strings.

---

§ 7

Component: Client `solariClient`

One per host, plus a watchdog. Inspects, catalogues, and reports local system attributes and the health of key software. This is the only component that must build for the full OS × architecture matrix, so its design is dominated by the platform-abstraction seam.

7.1 Responsibilities

- **Catalogue (slow-changing):** hostname/FQDN, OS & version, hardware, architecture (armv7/arm64/x86/x86_64/ppc/…), network addresses, total RAM/swap/disk. Sent on HELLO and when changed.
- **Sample (fast-changing):** per-core CPU load, RAM/swap used, free disk per mount, NIC throughput vs. capacity, USB-bus throughput vs. capacity.
- **Watch key software:** for a configured set of processes — running state, count of open files (incl. network sockets), RSS; plus log-file monitoring (size deltas + regex match counts).
- **Report:** send an `SCP_MSG_CLIENT_REPORT` with `ACK_REQ` to the authenticated control REQ/REP endpoint; keep it in the SQLite spool until the server ACK echoes its sequence and reporter session after durable persistence.

- **Watchdog:** a tiny sibling process that supervises the client, emits independent `SCP_MSG_WATCHDOG` heartbeats, and re-execs the client if it dies.

7.2 The portability seam — `platOS.h`

This is the entire portability strategy in one header. The client core calls *only* these functions; every OS-specific syscall lives in one of the three `plat/*.c` implementations selected at build time by CMake. Adding an OS means adding one `.c` file, nothing else.

▸ src/client/platOS.h

```c
/*
 * platOS.h — platform abstraction layer (PAL).
 * Exactly one of plat/platLinux.c, platMacos.c, platWindows.c is compiled,
 * chosen by CMake from the target. Every function returns solariStatus and
 * writes its out-param only on success. The client core includes this header
 * and nothing platform-specific; this is the seam that keeps the core ANSI-clean.
 */
#ifndef PLAT_OS_H
#define PLAT_OS_H
#include "solari/solariCommon.h"
#include "solari/solariMsg.h"

/* ---- identity & catalogue ---- */
solariStatus platHostFqdn  (char *out, size_t cap);
solariStatus platOsName    (char *out, size_t cap);   /* "Linux 6.1.0", "macOS 14.5", "Windows 11" */
solariStatus platArch      (char *out, size_t cap);   /* uname-style normalized token              */
solariStatus platCpuCount  (uint8_t *out);

/* ---- fast samples ---- */
/* Per-core load as permille (0..1000+). Caller passes array + capacity;
 * function writes coreCount entries. Linux: /proc/stat deltas; macOS:
 * host_processor_info; Windows: PDH counters. */
solariStatus platCpuLoad   (uint32_t *loadMilli, uint8_t cap, uint8_t *coreCount);
solariStatus platMemInfo   (uint64_t *ramUsedKb, uint64_t *ramTotalKb,
                            uint64_t *swapUsedKb, uint64_t *swapTotalKb);
solariStatus platDiskFree  (solariDiskEntry *disks, uint8_t cap, uint8_t *count);
solariStatus platNetIfaces (solariIfaceEntry *ifaces, uint8_t cap, uint8_t *count);
solariStatus platUsbThroughput(solariUsbEntry *bus, uint8_t cap, uint8_t *count);

/* ---- process & log watch ---- */
/* Resolve a watched process by name to pid+state+open-file/socket counts+RSS.
 * Linux: /proc/<pid>/{stat,fd}; macOS: libproc; Windows: ToolHelp + handles. */
solariStatus platProcInspect(const char *procName, solariProcEntry *out);
/* Stat a log file: current size (for delta vs last sample) + count lines
 * since last offset matching an optional POSIX regex. */
solariStatus platLogStat   (const char *path, const char *regex,
                            uint64_t *sizeNow, uint32_t *matchCount,
                            uint64_t *lastOffsetInOut);

/* ---- watchdog support ---- */
solariStatus platSpawnSelf (const char *argv0, int argc, char *const argv[]);
solariStatus platProcessAlive(int64_t pid, bool *alive);

#endif
```

PER-PLATFORM SOURCE MAP

Linux (`platLinux.c`): `/proc` + `getifaddrs` + `statvfs`; covers ARM32/ARM64/x86/x86_64/ppc uniformly. **macOS** (`platMacos.c`): `sysctl`, Mach `host_*` calls, `libproc`, `getfsstat`; universal (arm64 + x86_64). **Windows** (`platWindows.c`): Winsock for addresses, PDH for CPU/net counters, ToolHelp32 + `GetProcessHandleCount` for process inspection, `GetDiskFreeSpaceEx`. Built with mingw-w64 for x86_64.

7.3 Client core files & signatures

▸ src/client/clientCollect.c (header view)

```
/* Gather one full client report by calling the PAL. Returns a populated
 * solariClientReport. catalogueOnly=true skips fast samples (used on HELLO). */
solariStatus clientCollectReport(const clientConfig *cfg,
                                 bool catalogueOnly,
                                 solariClientReport *out);
```

▸ src/client/clientReport.c

```
/* Build the SCP frame from a report and either send it (active server,
 * discovered via who-is-primary) or spool it. Drains spool opportunistically. */
solariStatus clientReportSend(clientContext *ctx, const solariClientReport *rep);
solariStatus clientReportDrainSpool(clientContext *ctx);
```

▸ src/client/clientWatchdog.c

```
/* Watchdog entry point (separate process). Supervises the client pid, emits
 * SCP_MSG_WATCHDOG heartbeats every cfg->watchdogIntervalSec, and re-execs the
 * client via platSpawnSelf if platProcessAlive reports it gone. */
int clientWatchdogMain(const clientConfig *cfg, int64_t supervisedPid);
```

▸ src/client/main.c (control flow)

```
/* 1. parse args/config           5. loop:                              */
/* 2. solariLogInit                 - collect (fast)                     */
/* 3. fork watchdog sibling         - send/spool                         */
/* 4. HELLO -> WELCOME (schedule)     - honor any CONTROL on the back-chan */
/*                                    - sleep(scheduleInterval)          */
```

---

§ 8

Component: Remote Monitor `solariMonitor`

Distributed, interchangeable, fully redundant. Each monitor connects to TCP/UDP/ICMP endpoints of services and peers across the intranet, measuring reachability, response time, and link characteristics — then reports to the server. Targets Linux across the full architecture array; redundancy is by replication, not coordination.

8.1 Responsibilities

- **Service probing:** for each configured target, run TCP connect / UDP echo / ICMP echo probes, recording RTT, jitter, packet loss permille, throughput vs. capacity, and outcome (ok/timeout/refused/unreachable).
- **Stationary service metadata:** where a probe can cheaply learn it, capture service version/vendor/config banners (the "stationary" metadata in the brief).
- **Peer & network-wide metrics:** peer-to-peer reachability among monitors, transit time, bandwidth vs. capacity, connection speed, overhead, dropped packets.

- **Report & survive:** send `SCP_MSG_MONITOR_REPORT` with `ACK_REQ`; spool until durable server ACK. The current build still needs a verified survey-response sender before the monitor can answer `SCP_MSG_SURVEY` on demand.

8.2 Redundancy & fault tolerance model

Monitors are stateless w.r.t. each other except for a lightweight peer registry. The design goal — "interchangeable, fully redundant" — is met by *overlapping assignment*, not by leader election:

- The server assigns each probe target a **replication factor** (default 2). Targets are distributed across the monitor fleet by rendezvous (HRW) hashing of `(targetId, monitorNodeId)`, so each target is independently probed by its top-*k* monitors.
- If a monitor drops out, the HRW ranking simply re-selects the next monitor for affected targets on the next schedule tick — no failover handshake, no split-brain.
- The server deduplicates overlapping reports and treats divergence between vantage points as signal (e.g., target reachable from monitor A but not B localizes a network partition).
- Monitors exchange a tiny `peerAlive` gossip so each knows the live fleet membership for HRW; the server is the source of truth and corrects drift via `WELCOME`/`CONTROL`.

> WHY HRW (RENDEZVOUS) HASHING
>
> It gives consistent, low-churn target→monitor assignment with no coordinator and trivial recomputation when membership changes — exactly the property needed for "interchangeable and redundant." A monitor leaving reassigns only *its* targets; everyone else is untouched. This is materially simpler and more robust than consistent-hash rings or an elected scheduler for a fleet this size.

8.3 Probe engine & signatures

▸ src/monitor/probeNet.h

```
typedef enum { PROBE_ICMP = 1, PROBE_TCP = 2, PROBE_UDP = 3 } probeProto;
typedef enum {
    PROBE_OK = 0, PROBE_TIMEOUT, PROBE_REFUSED, PROBE_UNREACHABLE,
    PROBE_DNS_FAIL, PROBE_TLS_FAIL, PROBE_PROTO_ERR
} probeOutcome;

typedef struct {
    char       targetHost[256];
    uint16_t   dstPort;            /* 0 for ICMP                       */
    probeProto proto;
    uint16_t   count;              /* probes per round (for loss/jitter) */
    uint32_t   timeoutMs;
    const char *expectBanner;      /* optional substring match          */
} probeSpec;

/* Run one probe round per spec. Fills solariProbeResult (see 5.7). Uses the
 * monotonic clock for RTT. ICMP raw-socket privilege handling is isolated
 * here (Appendix C): tries raw socket, falls back to SOCK_DGRAM ICMP on
 * Linux where permitted, else marks PROBE_UNREACHABLE with detail. */
solariStatus probeRun(const probeSpec *spec, solariProbeResult *out);

/* Measure link characteristics to a peer monitor: transit time, one-way-ish
 * delay estimate, throughput vs. configured capacity, overhead. */
solariStatus probeLinkToPeer(uint64_t peerNodeId, const char *peerUrl,
                             solariLinkStat *out);
```

▸ src/monitor/monitorPeer.c & monitorSched.c

```c
/* HRW: does THIS monitor own this target at replication factor k? */
bool monitorOwnsTarget(uint64_t selfNodeId, const uint64_t *liveFleet,
                       size_t fleetLen, const char *targetId, uint8_t k);
/* Recompute the local probe schedule from config + current fleet membership. */
solariStatus monitorRebuildSchedule(monitorContext *ctx);
/* Gossip tick: send peerAlive, prune peers not heard from within TTL. */
solariStatus monitorPeerGossip(monitorContext *ctx);
```

---

§ 9

Component: Server `solariServer`

One primary, one failover. Collects client + monitor data, owns the master records, writes MariaDB, evaluates alerts, manages node config/deployment, and exposes the control surface the dashboard drives. Targets x86_64 and ARM64 only.

9.1 Subsystems

FILE	SUBSYSTEM	ROLE
`serverIngest.c`	Ingest	Validates report identities and role; writes transactional receipts with report data. Durable report ACKs are emitted by the control REP handler only after dispatch succeeds.
`serverLease.c`	Failover	Renews/claims the `serverLease` row; transitions active↔standby; answers `WHO_IS_PRIMARY`.
`serverDb.c`	Persistence	MariaDB Connector/C; prepared-statement upserts for master records, append-only history, lease ops. Connection pool.
`serverMaster.c`	Reconciliation	Merges overlapping monitor reports, maintains authoritative per-node/per-target current state, computes derived health.
`serverAlert.c`	Alerting	Evaluates threshold/transition rules; emits notifications (publishes to PUB socket → ntfy bridge).
`serverControl.c`	Control plane	Builds/sends `SCP_MSG_CONTROL`; tracks config epochs and deploy convergence per node.
`solariCtl.c`	Operator bridge	Listens on a local Unix socket; the PHP dashboard sends authenticated control requests here, and this helper (holding the server cert) translates them into SCP. Keeps cert material out of the web tier.

9.2 Failover signatures

▸ src/server/serverLease.c

```c
typedef enum { SRV_STANDBY, SRV_ACTIVE } serverRoleState;

/* Attempt to acquire/renew the lease. On success sets *state=SRV_ACTIVE and
 * advances epoch if it was a fresh claim. Single conditional UPDATE inside a
 * txn IS the election (see §2). Called every cfg->leaseRenewSec. */
solariStatus serverLeaseTick(serverContext *ctx, serverRoleState *state,
                             uint64_t *epoch);
/* Called on STANDBY->ACTIVE: bind ingest sockets, resume DB writes.
 * On ACTIVE->STANDBY: unbind, flush, go read-only. Idempotent. */
solariStatus serverApplyRole(serverContext *ctx, serverRoleState newState);
```

▸ src/server/serverIngest.c

```c
/* Validate a received frame: header sanity, proto version, crc already
 * checked by parse; here enforce that msgType is legal for the presenting
 * cert role, and dedup. Returns ERR_AUTH_ROLE on violation. */
solariStatus serverIngestValidate(serverContext *ctx,
                                  const solariFrameHeader *hdr,
                                  const char *peerCertCn);
/* Dispatch one validated frame to the right handler (report->db, survey
 * response->reconcile, control-result->convergence tracker, etc.). */
solariStatus serverIngestDispatch(serverContext *ctx,
                                  const solariFrameHeader *hdr,
                                  const uint8_t *payload, size_t len);
```

▸ src/server/serverDb.c (representative writer)

```c
/* Upsert current state + append history in one transaction. Uses prepared
 * statements bound from the parsed report. All time stored UTC. */
solariStatus serverDbWriteClientReport(serverContext *ctx, uint64_t nodeId,
                                       const solariClientReport *rep);
solariStatus serverDbWriteMonitorReport(serverContext *ctx, uint64_t nodeId,
                                        const solariMonitorReport *rep);
```

---

§ 10

Database schema (MariaDB)

The contract boundary between the C server and the PHP dashboard. Current-state tables are upserted; history tables are append-only and partitioned by time for retention pruning.

▸ db/schema.sql

```sql
CREATE TABLE node (
  nodeId        BIGINT UNSIGNED PRIMARY KEY,   -- FNV-1a-64 (§5.5)
  role          ENUM('client','monitor','server') NOT NULL,
  hostFqdn      VARCHAR(255) NOT NULL,
  certCn        VARCHAR(255) NOT NULL,
  osName        VARCHAR(64),
  arch          VARCHAR(16),
  enrolledAt    DATETIME NOT NULL,
  lastSeenAt    DATETIME,
  configEpoch   BIGINT UNSIGNED NOT NULL DEFAULT 0,
  state         ENUM('up','degraded','down','unknown') NOT NULL DEFAULT 'unknown',
  INDEX(role), INDEX(lastSeenAt)
) ENGINE=InnoDB;

-- current host metrics (one row per client node, upserted)
CREATE TABLE hostCurrent (
  nodeId        BIGINT UNSIGNED PRIMARY KEY,
  sampledAt     DATETIME NOT NULL,
  cpuLoadMilli  JSON,                  -- per-core array
  ramUsedKb     BIGINT UNSIGNED, ramTotalKb BIGINT UNSIGNED,
  swapUsedKb    BIGINT UNSIGNED, swapTotalKb BIGINT UNSIGNED,
  disks         JSON, ifaces JSON, usbBuses JSON,
  CONSTRAINT FK_hc_node FOREIGN KEY(nodeId) REFERENCES node(nodeId) ON DELETE CASCADE
) ENGINE=InnoDB;

-- append-only host history (partitioned monthly for retention)
CREATE TABLE hostHistory (
  id            BIGINT UNSIGNED AUTO_INCREMENT,
  nodeId        BIGINT UNSIGNED NOT NULL,
  sampledAt     DATETIME NOT NULL,
  cpuAvgMilli   INT UNSIGNED, ramUsedKb BIGINT UNSIGNED,
  swapUsedKb    BIGINT UNSIGNED, diskMinFreePct SMALLINT,
  PRIMARY KEY(id, sampledAt), INDEX(nodeId, sampledAt)
) ENGINE=InnoDB
  PARTITION BY RANGE (TO_DAYS(sampledAt)) ( /* monthly partitions, auto-managed */ );

-- watched processes (current), keyed per node+process
CREATE TABLE procCurrent (
  nodeId   BIGINT UNSIGNED NOT NULL, procName VARCHAR(128) NOT NULL,
  pid INT, runState CHAR(1), nFiles INT, nSockets INT, rssKb BIGINT UNSIGNED,
  sampledAt DATETIME NOT NULL,
  PRIMARY KEY(nodeId, procName)
) ENGINE=InnoDB;

-- probe targets & their current measured state (from monitors)
CREATE TABLE probeTarget (
  targetId   VARCHAR(128) PRIMARY KEY,    -- stable id, e.g. "tcp:hydrogen:443"
  host       VARCHAR(255), port SMALLINT UNSIGNED,
  proto      ENUM('icmp','tcp','udp') NOT NULL,
  replFactor TINYINT NOT NULL DEFAULT 2,
  label      VARCHAR(128)
) ENGINE=InnoDB;

CREATE TABLE probeCurrent (
  targetId    VARCHAR(128) NOT NULL,
  monitorNode BIGINT UNSIGNED NOT NULL,     -- which vantage reported
  outcome     ENUM('ok','timeout','refused','unreachable','dns_fail','tls_fail','proto_err'),
  rttMicros   INT UNSIGNED, jitterMicros INT UNSIGNED,
  lossPermille SMALLINT UNSIGNED, throughputKbps INT UNSIGNED,
  serviceMeta JSON, sampledAt DATETIME NOT NULL,
  PRIMARY KEY(targetId, monitorNode)
) ENGINE=InnoDB;

CREATE TABLE probeHistory ( /* append-only, partitioned monthly, same cols + id */
  id BIGINT UNSIGNED AUTO_INCREMENT, targetId VARCHAR(128),
  monitorNode BIGINT UNSIGNED, outcome VARCHAR(16),
  rttMicros INT UNSIGNED, lossPermille SMALLINT UNSIGNED, sampledAt DATETIME,
  PRIMARY KEY(id, sampledAt), INDEX(targetId, sampledAt)
) ENGINE=InnoDB PARTITION BY RANGE(TO_DAYS(sampledAt)) ();

-- failover coordination: exactly one row (id=1)
CREATE TABLE serverLease (
  id          TINYINT PRIMARY KEY DEFAULT 1,
  leaseHolder BIGINT UNSIGNED,           -- server nodeId
  leaseEpoch  BIGINT UNSIGNED NOT NULL DEFAULT 0,
```

```
  expiresAt   DATETIME(3) NOT NULL,
  CHECK (id = 1)
) ENGINE=InnoDB;

-- alert rules & firing log
CREATE TABLE alertRule (
  ruleId   INT AUTO_INCREMENT PRIMARY KEY,
  scope    ENUM('host','probe') NOT NULL,
  metric   VARCHAR(64) NOT NULL,        -- e.g. "cpuAvgMilli","lossPermille"
  op       ENUM('gt','lt','eq','transition') NOT NULL,
  threshold DOUBLE, forSeconds INT, severity ENUM('info','warn','crit'),
  enabled  BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;

CREATE TABLE alertEvent (
  eventId  BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  ruleId   INT, nodeId BIGINT UNSIGNED, targetId VARCHAR(128),
  firedAt  DATETIME NOT NULL, clearedAt DATETIME,
  severity ENUM('info','warn','crit'), detail VARCHAR(512),
  INDEX(firedAt), INDEX(nodeId)
) ENGINE=InnoDB;

-- per-node config & deploy convergence tracking
CREATE TABLE nodeConfig (
  nodeId      BIGINT UNSIGNED PRIMARY KEY,
  targetEpoch BIGINT UNSIGNED NOT NULL, appliedEpoch BIGINT UNSIGNED,
  configBlob  JSON, lastDirectiveAt DATETIME, lastResult VARCHAR(128)
) ENGINE=InnoDB;
```

---

§ 11

Dashboard data contract (PHP / Apache)

The visual design is produced separately in Claude Design. This section fixes only what the C system guarantees and what the dashboard may rely on, so the two compose without renegotiation. PHP reads MariaDB via PDO; control actions round-trip through `solariCtl`.

11.1 Architecture

- **Read path:** PHP 8.x reads current/history tables via PDO (prepared statements only). No PHP writes to monitoring tables — those are the C server's exclusively.
- **Live updates:** browser polls the JSON API (default 5 s) *or* subscribes to a Server-Sent Events endpoint (`/api/stream`) that the PHP layer feeds from the server's PUB socket via the `solariCtl` bridge. (SSE, not WebSockets — no Node, no long-lived PHP worker beyond a simple bridge.)
- **Control path:** operator POSTs (e.g. "survey now", "push config") authenticate at PHP, then PHP calls `solariCtl` over its Unix socket; the helper emits the SCP control frame. Cert material never enters the web tier.

11.2 REST/JSON API (the contract Claude Design binds to)

METHOD & PATH	RETURNS / ACCEPTS	NOTES
`GET /api/nodes`	Array of node summaries: `{nodeId, role, hostFqdn, state, lastSeenAt, arch, osName}`	The fleet roster + rollup state.
`GET /api/nodes/{id}`	Full current host metrics + watched procs + recent alerts for one node.	Node detail view.
`GET /api/nodes/{id}/history?metric=&from=&to=`	Time series for one metric (downsampled server-side).	Charts.
`GET /api/probes`	Probe targets with per-vantage current state (the multi-monitor view).	Reachability matrix.
`GET /api/probes/{targetId}/history?…`	RTT/loss time series per vantage.	Latency charts.
`GET /api/alerts?status=active|all`	Alert events with rule + node context.	Alert panel.
`GET /api/topology`	Nodes + monitor→target assignment edges (HRW result) for a live map.	Feeds the C2 map view.
`GET /api/stream`	SSE: pushes `nodeStateChange`, `alertFired`, `probeUpdate` events.	Live tiles without polling.
`POST /api/control/survey`	Body `{scope:"all"|nodeId}`; triggers immediate survey.	Auth required → solariCtl.
`POST /api/control/config`	Body `{nodeId, configBlob}`; stages config push.	Auth + RBAC (operator role).
`GET /api/server/status`	`{primary:nodeId, failover:nodeId, leaseEpoch, expiresAt}`	Failover state banner.

All responses are JSON with a stable envelope `{ "ok": true, "data": …, "ts": "ISO-8601" }` or `{ "ok": false, "error": {code, message} }`. All timestamps ISO-8601 UTC. Pagination via `?limit=&cursor=`. These shapes mirror the JSON projection in §5.10 so a single mental model spans wire → DB → API.

11.3 Auth & RBAC

- Session-based login (PHP sessions, Argon2id password hashing). Optional reverse-proxy SSO if the operator fronts it.
- Two roles: **viewer** (all GET) and **operator** (GET + control POSTs). Role checked server-side on every control endpoint.
- CSRF tokens on all POSTs; control POSTs additionally re-verified by `solariCtl` before any SCP frame is emitted.

---

§ 12

Build system & cross-compilation

CMake orchestrates the whole tree and every cross-architecture target; a thin Makefile wraps the common invocations. Third-party deps build from vendored source so the matrix stays hermetic.

12.1 Top-level CMake (shape)

▸ CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(solarinet C)
set(CMAKE_C_STANDARD 99)
set(CMAKE_C_STANDARD_REQUIRED ON)
add_compile_options(-Wall -Wextra -Wpedantic -Werror=implicit-function-declaration)

# vendored deps built from source (hermetic, cross-friendly)
add_subdirectory(third_party/mbedtls)
add_subdirectory(third_party/nng)
add_subdirectory(third_party/cJSON)
# sqlite + unity added as object libs

add_subdirectory(lib/solari)        # libsolari (static)
add_subdirectory(src/client)        # solariClient   (all targets)
add_subdirectory(src/monitor)       # solariMonitor  (linux, all arch)
if(SOLARI_BUILD_SERVER)               # server only on x86_64/arm64
  add_subdirectory(src/server)
endif()
if(SOLARI_BUILD_TESTS)
  enable_testing()
  add_subdirectory(tests)
endif()
```

12.2 Per-component target gating

COMPONENT	OS TARGETS	ARCHITECTURES
`solariClient`	Linux, macOS, Windows	armv7 (ARM32), arm64, x86, x86_64 (+ppc where toolchain exists)
`solariMonitor`	Linux	armv7, arm64, x86, x86_64 (full array)
`solariServer`	Linux	x86_64, arm64 only (gated by `SOLARI_BUILD_SERVER`)

12.3 Toolchain file (representative)

▸ `cmake/toolchains/linux-arm64.cmake`

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR aarch64)
set(CMAKE_C_COMPILER aarch64-linux-gnu-gcc)
set(CMAKE_FIND_ROOT_PATH /usr/aarch64-linux-gnu)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
```

12.4 Makefile wrapper

▸ `Makefile`

```
# convenience over CMake; muscle-memory targets
BUILD ?= build

all: client monitor server

client:
	cmake -S . -B $(BUILD) -DSOLARI_BUILD_SERVER=OFF && cmake --build $(BUILD) --target solariClient

server:
	cmake -S . -B $(BUILD) -DSOLARI_BUILD_SERVER=ON && cmake --build $(BUILD) --target solariServer

cross-arm64:
	cmake -S . -B build-arm64 \
	  -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/linux-arm64.cmake \
	  -DSOLARI_BUILD_SERVER=ON && cmake --build build-arm64

cross-arm32:
	cmake -S . -B build-arm32 \
	  -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/linux-arm32.cmake \
	  -DSOLARI_BUILD_SERVER=OFF && cmake --build build-arm32 --target solariClient solariMonitor

windows:
	cmake -S . -B build-win \
	  -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/windows-x86_64.cmake \
	  -DSOLARI_BUILD_SERVER=OFF && cmake --build build-win --target solariClient

test:
	cmake -S . -B $(BUILD) -DSOLARI_BUILD_TESTS=ON && cmake --build $(BUILD) && ctest --test-dir $(BUILD) --
output-on-failure

clean:
	rm -rf build build-*
```

---

§ 13

Configuration formats

Plain `key = value` files, `#` comments, sections in `[brackets]`. A JSON overlay (pushed by the server as a config blob) is merged on top at load, so operators get human-editable files *and* central control.

▸ config/client.conf.sample

▸ config/monitor.conf.sample

```
[identity]
nodeId      = # filled at enrollment
hostFqdn    = hydrogen.akoria.net

[server]
primaryUrl  = tls+tcp://argon.akoria.net:7701
failoverUrl = tls+tcp://neon.akoria.net:7701
# Optional if not the matching primaryUrl port + 1
(default 7702)
reportAckUrl = tls+tcp://argon.akoria.net:7702

[tls]
caFile   = /etc/solari/ca.pem
certFile = /etc/solari/node.pem
keyFile  = /etc/solari/node.key

[schedule]
sampleIntervalSec   = 15
watchdogIntervalSec = 5

[watch]
process = apache2
process = mariadbd
logfile = /var/log/syslog : (error|fail)
spoolDb = /var/lib/solari/spool.db
```

```
[identity]
nodeId   = # enrollment

[server]
primaryUrl  = tls+tcp://argon.akoria.net:7701
failoverUrl = tls+tcp://neon.akoria.net:7701
# Optional if not the matching primaryUrl port + 1
(default 7702)
reportAckUrl = tls+tcp://argon.akoria.net:7702

[probe]
target = tcp:hydrogen.akoria.net:443 : web
target = icmp:10.42.0.1 : gateway
target = udp:10.42.0.53:53 : dns
roundIntervalSec = 30
probesPerRound   = 5
replFactorHint   = 2

[peer]
gossipIntervalSec = 20
peerTtlSec        = 90
```

▸ config/server.conf.sample

```
[identity]
nodeId = # enrollment; both servers distinct
[bind]
ingestUrl = tls+tcp://0.0.0.0:7701
surveyUrl = tls+tcp://0.0.0.0:7702
pubUrl    = tls+tcp://0.0.0.0:7703
ctlSocket = /run/solari/solariCtl.sock
[lease]
leaseRenewSec = 5
leaseTtlSec   = 15
[db]
host = 127.0.0.1
name = solarinet
user = solari
# password via SOLARI_DB_PASS env, never in file
poolSize = 8
[retention]
historyDays = 90
```

§ 14

Testing strategy

LAYER	TOOLING	WHAT IT PROVES
Unit	Unity + CMock, per module	Frame pack/unpack round-trips on both endiannesses (simulated); TLV append/read symmetry; CRC correctness; config parsing; HRW determinism & low churn; lease state machine transitions.
Protocol conformance	Golden-frame fixtures (byte-exact)	A captured frame decodes to the expected struct and re-encodes byte-identically. Guards wire stability across refactors. Fixtures double as the JSON-projection oracle.
Integration	Docker-composed Linux fleet (server + MariaDB + N monitors + N clients)	End-to-end: client sample → DB row; monitor probe → reachability matrix; survey → immediate round; control push → epoch convergence.
Fault injection	`tests/fault/` harnesses	Kill the active server → failover promotes < lease TTL, no data loss (spool drains). Partition a monitor → HRW reassigns its targets. Sever server link → clients spool then drain. Corrupt a frame → CRC rejects, no crash.
Cross-build CI	CMake matrix (qemu-user for non-native arch)	Client builds & passes smoke tests on armv7/arm64/x86_64 Linux, mingw Windows, macOS (native runner); monitor on the Linux arch array; server on x86_64/arm64.

> AGENT TEST MANDATE
>
> Every public `libsolari` function ships with at least one Unity test before its first consumer is wired up. The protocol-conformance golden fixtures are written *first* (from §5) and treated as immutable for a given `SCP_PROTO_VERSION` — they are the definition of "correct on the wire."

---

§ 15

Deployment & operations

15.1 Enrollment flow (one-time per node)

1. Operator generates an enrollment token on the server (short-TTL, single-use), delivered out-of-band.
2. Node generates a keypair + CSR, presents token + CSR to the server's enrollment endpoint.
3. Server (using the internal CA / step-ca root) signs the cert with a CN encoding role + FQDN, registers the node row, returns the signed cert + CA root.
4. Node persists cert/key/CA, computes its `nodeId`, writes it to config, and connects normally thereafter.

15.2 systemd units (Linux)

▸ `deploy/systemd/solari-client.service`

```
[Unit]
Description=SolariNet Client
After=network-online.target
Wants=network-online.target
[Service]
Type=simple
ExecStart=/usr/local/bin/solariClient --config /etc/solari/client.conf
Restart=always
RestartSec=2
User=solari
AmbientCapabilities=CAP_NET_RAW   # client needs none; monitor unit adds this for ICMP
[Install]
WantedBy=multi-user.target
```

The watchdog is spawned by the client itself (§7.3), so systemd supervises the client and the client supervises its watchdog — defense in depth without a second unit. The monitor unit is analogous but adds `CAP_NET_RAW` for ICMP. Two server units (`solari-server.service`) run on the primary and failover hosts; both start identically and sort out active/standby via the lease.

15.3 Upgrade / deploy

Operator stages a new binary; server issues `CTRL_DEPLOY` (with SHA-256) to a node, which fetches, verifies, and stages it, then `CTRL_RESTART` triggers a watchdog-mediated re-exec. Rollout is staged (canary node → cohort → fleet) and tracked via `nodeConfig.appliedEpoch` convergence visible on the dashboard.

---

§ 16

Phased project plan & task breakdown

The execution worklist. Phases are ordered so each builds only on what precedes it; an agent can work top-to-bottom. Each task names the file(s) and the §-reference that specifies it.

PHASE 0 · FOUNDATION NO DEPS

Repo, build skeleton, vendored deps

- Initialize git repo with the §4 layout; add `.gitignore`, `LICENSE`, self-contained `README.html`.
- Vendor & pin nng, mbedTLS, SQLite amalgamation, cJSON, Unity under `third_party/`.
- Top-level `CMakeLists.txt` + `Makefile` + the five toolchain files (§12). Prove an empty `libsolari` + a "hello" client cross-build for arm64 and mingw.

PHASE 1 · LIBSOLARI CORE DEPENDS: P0

Common, error, time, log, crypto, framing, TLV

- `solariCommon.h`, `solariError.{h,c}` (Appendix A enum + strings).
- `solariTime.c`, `solariLog.c`, `solariCrypto.c` (FNV-1a-64, CRC-32, SHA-256, cert CN).
- `solariFrame.c` + `solariTlv.c` — pack/unpack/build/parse (§§5,6). **Write golden-frame fixtures & Unity tests first.**
- `solariMsg.c` typed builders/parsers for every message type (§5.4).

PHASE 2 · LIBSOLARI I/O DEPENDS: P1

Transport, config, spool

- `solariNet.c` — nng wrapper + mbedTLS for all patterns (§6.5). Loopback round-trip test over TLS.
- `solariConfig.c` — `key=value` parser + JSON overlay merge (§13).
- `solariSpool.c` — SQLite store-and-forward (§6.6) with push/peek/ack/nack + backoff. Fault test: enqueue while "offline," drain on "reconnect."

PHASE 3 · CLIENT + PAL DEPENDS: P2

solariClient across the OS matrix

- `platOS.h` contract; then `platLinux.c` first (reference impl), `platMacos.c`, `platWindows.c` (§7.2).
- `clientCollect.c`, `clientReport.c`, `clientWatchdog.c`, `main.c` (§7.3).
- HELLO/WELCOME handshake; sample loop; spool-on-failure; honor CONTROL.
- Cross-build & smoke-test on arm64 + mingw; native macOS run.

PHASE 4 · REMOTE MONITOR DEPENDS: P2

solariMonitor probe engine + redundancy

- `probeNet.c` — TCP/UDP/ICMP probes with RTT/jitter/loss (§8.3); ICMP privilege handling per Appendix C.
- `monitorPeer.c` (HRW ownership, gossip) + `monitorSched.c` (schedule rebuild) (§8.2).
- `monitorReport.c` + `main.c`; survey responder.
- Fault test: drop a monitor, confirm HRW reassignment with minimal churn.

PHASE 5 · SERVER DEPENDS: P1–P4, SCHEMA

solariServer ingest, failover, persistence

- Apply the ordered monitoring migrations with `tools/migrate.py`; `db/schema.sql` is a generated reference snapshot. `serverDb.c` provides MariaDB Connector/C, prepared statements, and the pool.
- `serverLease.c` (election) + `serverIngest.c` (validate/dedup/dispatch) + `serverMaster.c` (reconcile overlaps).
- `serverAlert.c` (rule eval → PUB) + `serverControl.c` (config/deploy push, epoch tracking).
- `solariCtl.c` Unix-socket operator bridge.
- Fault test: kill active server → failover < TTL, zero loss.

PHASE 6 · DASHBOARD DATA LAYER DEPENDS: P5

PHP/Apache API (contract only; visuals via Claude Design)

- PDO data-access layer (read-only on monitoring tables); auth + RBAC + CSRF (§11.3).
- Implement every endpoint in §11.2 returning the stable envelope; SSE bridge from server PUB via `solariCtl`.
- Hand the endpoint contract to Claude Design for the C2 UI; wire its components to these endpoints.

PHASE 7 · HARDENING & OPS DEPENDS: ALL

Enrollment, packaging, CI, docs

- Enrollment flow + scripts (§15.1); systemd units; retention/partition maintenance job.
- Full cross-build CI matrix (qemu-user) + integration + fault suites green.
- Self-contained HTML operator & developer docs in `docs/`.

Dependency graph (at a glance)



---

§ 17

Coding standards

- **Language:** ISO C99, functions only. No C++, no OOP emulation beyond the documented opaque-handle pattern (a `typedef struct foo foo;` with create/use/destroy functions). No global mutable state; thread state travels in context structs passed by pointer.

- **Naming:** `camelCase` for functions, variables, struct fields, and typedef'd types; `UPPER_SNAKE` for macros and enum constants; module-prefixed public names (`solariFrameParse`, `platCpuLoad`, `monitorOwnsTarget`). Descriptive but not unwieldy.
- **Brace style:** K&R (opening brace on the same line; `else` cuddled). Four-space indent, no tabs in C.
- **Returns & errors:** functions return `solariStatus`; out-params written only on success unless documented. No `errno`-style hidden state across the API. Every error path logs at an appropriate level and propagates a code from Appendix A.
- **Memory:** explicit ownership documented in each function's doc-comment; matching `*Open/*Close`, `*Init/*Destroy` pairs; zero-copy reads alias caller buffers and say so.
- **Doc-comments:** every public function has a block comment stating purpose, ownership/lifetime of pointers, success/failure semantics, and platform notes where relevant (as shown throughout §§6–9).
- **Portability:** no syscalls outside the PAL; fixed-width `<stdint.h>` types on the wire; never assume endianness or struct padding — pack field-by-field.
- **Git:** conventional-commit style messages; the monorepo lives in a local git repo, with publishable components mirrored to GitHub at the operator's discretion.

---

§ 18

Appendices

Appendix A — Error codes (`solariError.h`)

CODE	VALUE	MEANING
`SOLARI_OK`	0	Success.
`ERR_INVALID_ARG`	-1	Bad argument / null where non-null required.
`ERR_BUFFER_FULL`	-2	Write would overflow caller buffer.
`ERR_FRAME_INCOMPLETE`	-10	Stream holds less than one full frame.
`ERR_FRAME_BAD_MAGIC`	-11	Sentinel mismatch.
`ERR_FRAME_BAD_CRC`	-12	CRC-32 verification failed.
`ERR_PROTO_VERSION`	-13	Unsupported protocol version.
`ERR_FRAME_BAD_ENVELOPE`	-14	Nonzero reserved byte, unsupported flag, malformed TLV sequence, or TLV count mismatch.
`ERR_TLV_END`	-20	Reader reached end of payload (loop terminator, not fatal).
`ERR_TLV_TRUNCATED`	-21	TLV length exceeds remaining bytes.
`ERR_CONN_RETRY`	-30	Transient send/recv failure; caller should spool/retry.
`ERR_CONN_FATAL`	-31	Unrecoverable transport error.
`ERR_TLS`	-32	TLS handshake/verify failure.
`ERR_AUTH_ROLE`	-40	Message type illegal for presenting cert role.
`ERR_DB`	-50	Database error (detail logged).
`ERR_LEASE_LOST`	-51	Active server failed to renew lease; must demote.
`ERR_UNKNOWN_MSG`	-60	Unrecognized message type that must be understood.
`ERR_REJECTED`	-61	Well-formed operation rejected by current authoritative state.
`ERR_PLATFORM`	-70	PAL call failed (OS-specific detail logged).

Appendix B — Default ports

PORT	PURPOSE	PATTERN
7701/tcp	Best-effort server ingest (watchdog, adverts, control results)	PULL ← PUSH, TLS
7702/tcp	Survey, HELLO/primary lookup, durable report ACK	SURVEYOR/RESPONDENT + REQ/REP, TLS
7703/tcp	Event publish (dashboard/notifiers)	PUB → SUB, TLS
unix sock	`solariCtl` operator bridge	REQ/REP, local only

Appendix C — ICMP portability notes

ICMP echo needs either a raw socket (root / `CAP_NET_RAW`) or, on Linux, the unprivileged `SOCK_DGRAM/IPPROTO_ICMP` path gated by `net.ipv4.ping_group_range`. `probeRun` tries, in order: (1) `SOCK_DGRAM` ICMP if the gid is permitted; (2) raw socket if the process holds `CAP_NET_RAW` (the monitor systemd unit grants it ambiently); (3) otherwise records `PROBE_UNREACHABLE` with a diagnostic detail rather than crashing. Since the monitor targets Linux only, the macOS/Windows ICMP idiosyncrasies don't apply to it; the client never sends ICMP.

Appendix D — Glossary

SCP — SolariNet Control Protocol (§5). · **PAL** — Platform Abstraction Layer (`platOS.h`, §7.2). · **HRW** — Highest Random Weight (rendezvous) hashing, the monitor's coordinator-free target-assignment scheme (§8.2). · **TLV** — Type-Length-Value, the extensible payload element (§5.6). · **Lease** — the single-row DB mutex that elects the active server (§2). · **Spool** — local SQLite store-and-forward queue enabling fault tolerance (§6.6). · **C2** — Command & Control (the dashboard's role).

This document records the architecture and implementation contracts. The active stabilization workflow and dated baseline linked above define current work and release status. Where implementation details differ, reconcile them against those current artifacts and update this reference.